

An Analysis of Naive Bayes classifier

Naive Bayes classifier

$$P(C_k | x) \propto P(C_k) \prod_{i=1}^n P(x_i | C_k)$$

Naive Bayes classifier -- Part 1

$p(\text{height} = \text{male}) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{(6-\mu)^2}{2\sigma^2}\right) \approx 1.5789$,
$$p(\text{height} \mid \text{male}) = \frac{1}{\sqrt{2\pi} \sigma} \exp\left(-\frac{(6-\mu)^2}{2\sigma^2}\right) \approx 1.5789,$$

Naive Bayes classifier -- Part 2

Probabilistic model Abstractly, naive Bayes is a conditional probability model: it assigns probabilities $p(C_k \mid x_1, \dots, x_n)$ for each of the K possible outcomes or classes C_k given a problem instance to be classified, represented by a vector $x = (x_1, \dots, x_n)$ encoding some n features (independent variables). The problem with the above formulation is that if the number of features n is large or if a feature can take on a large number of values, then basing such a model on probability tables is infeasible. The model must therefore be reformulated to make it more tractable. Using Bayes' theorem, the conditional probability can be decomposed as:

Naive Bayes classifier -- Part 3

Document classification Here is a worked example of naive Bayesian classification to the document classification problem. Consider the problem of classifying documents by their content, for example into spam and non-spam e-mails. Imagine that documents are drawn from a number of classes of documents which can be modeled as sets of words where the (independent) probability that the i -th word of a given document occurs in a document from class C can be written as

Naive Bayes classifier -- Part 4

The left-hand side of this equation is the log-odds, or logit, the quantity predicted by the linear model that underlies logistic regression. Since naive Bayes is also a linear model for the two "discrete" event models, it can be reparametrised as a linear function $b + w \cdot x > 0$. Obtaining the probabilities is then a matter of applying the logistic function to $b + w \cdot x$, or in the multiclass case, the softmax function. Discriminative classifiers have lower asymptotic error than

generative ones; however, research by Ng and Jordan has shown that in some practical cases naive Bayes can outperform logistic regression because it reaches its asymptotic error faster.

Naive Bayes classifier -- Part 5

prior for a given class = no. of samples in that class / total no. of samples
$$\text{prior for a given class} = \frac{\text{no. of samples in that class}}{\text{total no. of samples}}$$

Naive Bayes classifier -- Part 6

$$p(x \mid C_k) = \prod_{i=1}^n p_{ki}^{x_i} (1 - p_{ki})^{(1 - x_i)}$$

$$p(\mathbf{x} \mid C_k) = \prod_{i=1}^n p_{ki}^{x_i} (1 - p_{ki})^{(1 - x_i)}$$

Naive Bayes classifier -- Part 7

Other heuristics "Neutral" words like "the", "a", "some", or "is" (in English), or their equivalents in other languages, can be ignored. These are also known as Stop words. More generally, some bayesian filtering filters simply ignore all the words which have a spamicity next to 0.5, as they contribute little to a good decision. The words taken into consideration are those whose spamicity is next to 0.0 (distinctive signs of legitimate messages), or next to 1.0 (distinctive signs of spam). A method can be for example to keep only those ten words, in the examined message, which have the greatest absolute value $|0.5 - p_i|$. Some software products take into account the fact that a given word appears several times in the examined message, others don't. Some software products use patterns (sequences of words) instead of isolated natural languages words. For example, with a "context window" of four words, they compute the spamicity of "Viagra is good for", instead of computing the spamicities of "Viagra", "is", "good", and "for". This method gives more sensitivity to context and eliminates the Bayesian noise better, at the expense of a bigger database.

Naive Bayes classifier -- Part 8

$$p(x \mid C_k) = \frac{(\sum_{i=1}^n x_i)!}{\prod_{i=1}^n x_i! \prod_{i=1}^n p_{ki}^{x_i}}$$

$$p(\mathbf{x} \mid C_k) = \frac{(\sum_{i=1}^n x_i)!}{\prod_{i=1}^n x_i! \prod_{i=1}^n p_{ki}^{x_i}}$$

Naive Bayes classifier -- Part 9

Multinomial naive Bayes With a multinomial event model, samples (feature vectors) represent the frequencies with which certain events have been generated by a multinomial $(p_1, \dots, p_n)$ where p_i

An Analysis of Naive Bayes classifier

Naive Bayes classifier

p_i is the probability that event i occurs (or K such multinomials in the multiclass case). A feature vector $\mathbf{x} = (x_1, \dots, x_n)$ is then a histogram, with x_i counting the number of times event i was observed in a particular instance. This is the event model typically used for document classification, with events representing the occurrence of a word in a single document (see bag of words assumption). The likelihood of observing a histogram $\mathbf{x}$ is given by:

Naive Bayes classifier -- Part 10

Gaussian naive Bayes When dealing with continuous data, a typical assumption is that the continuous values associated with each class are distributed according to a normal (or Gaussian) distribution. For example, suppose the training data contains a continuous attribute, x . The data is first segmented by the class, and then the mean and variance of x is computed in each class. Let μ_k be the mean of the values in x associated with class C_k , and let σ_k^2 be the Bessel corrected variance of the values in x associated with class C_k . Suppose one has collected some observation value v . Then, the probability density of v given a class C_k , i.e., $p(x = v \mid C_k)$, can be computed by plugging v into the equation for a normal distribution parameterized by μ_k and σ_k^2 . Formally,